System-2 Course: Session 4
Neuro-Symbolic VQA: Disentangling Reasoning from Vision and Language Understanding

Farzan Rahmani - 403210725

Spring 2025

Contents

1 Introduction **2**

2 The CLEVR Diagnostic Dataset **2**
2.1 Motivation . . . 2
2.2 Design Principles . . . 2
2.3 Dataset Statistics . . . 3
2.4 Analysis of VQA Models on CLEVR . . . 3

3 Core Idea: Disentangling Perception from Reasoning **3**

4 Architecture Overview **4**
4.1 Scene Parser (De-renderer) . . . 4
4.2 Question Parser (Program Generator) . . . 4
4.3 Program Executor . . . 5

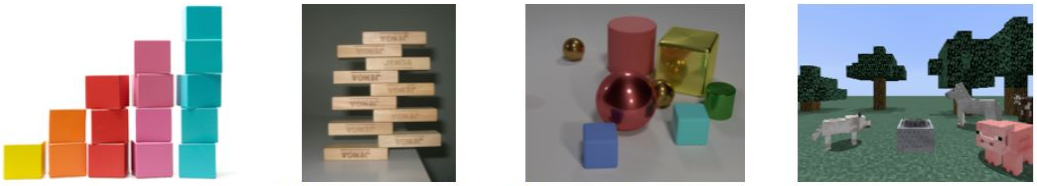
5 Question Parser Architecture Details **5**
5.1 Encoder (Bidirectional LSTM) . . . 5
5.2 Decoder (LSTM with Attention) . . . 5

6 Training the Modules **6**
6.1 Scene Parser Training . . . 6
6.2 Question Parser Training . . . 6

7 REINFORCE Algorithm Details **7**
7.1 Question Parser Training . . . 7
7.2 REINFORCE Fundamentals . . . 7
7.3 Gradient Derivation . . . 7
7.4 Trajectory Probability . . . 8
7.5 Final Gradient Expression . . . 8
7.6 Monte Carlo Implementation . . . 8
7.7 Parameter Update . . . 8
7.8 Algorithm Steps . . . 8
7.9 Variance Reduction . . . 9

8 Key Advantages and Results **9**
8.1 Robustness to Long Program Traces and Complex Reasoning . . . 9
8.2 Data and Memory Efficiency . . . 9
8.3 Transparency and Interpretability . . . 9
8.4 Generalization Capabilities . . . 9

9 Conclusion and Future Directions **11**



How many blocks are on the right of the three-level tower? | Will the block tower fall if the top block is removed? | What is the shape of the object closest to the large cylinder? | Are there more trees than animals?

Figure 1: Human reasoning is interpretable and disentangled: we first draw abstract knowledge of the scene via visual perception and then perform logic reasoning on it. This enables compositional, accurate, and generalizable reasoning in rich visual contexts.

1 Introduction

Remark 1. *This lecture note provides a comprehensive overview of Neuro-Symbolic Visual Question Answering (VQA), drawing upon the lecture slides of the course "CS 957, System-2 AI", the paper "Neural-Symbolic VQA: Disentangling Reasoning from Vision and Language Understanding" by Yi et al. [3], and the paper "CLEVR: A Diagnostic Dataset for Compositional Language and Elementary Visual Reasoning" by Johnson et al. [1]. This approach emphasizes the separation of perception and reasoning for more robust and interpretable VQA systems.*

Some examples of the VQA task are in Figure 1 which need to be answered in natural language text.

2 The CLEVR Diagnostic Dataset

The CLEVR dataset plays a crucial role in evaluating and understanding the capabilities of VQA systems, including neuro-symbolic ones. A field guide to the CLEVR dataset is depicted in the Figure 2.

2.1 Motivation

- Existing VQA benchmarks often contain biases that models can exploit without performing true visual reasoning.
- They conflate multiple sources of error.
- CLEVR was designed as a diagnostic dataset to test a range of visual reasoning abilities with minimal biases and detailed annotations.
- The name is a homage to Clever Hans, illustrating the danger of systems superficially solving tasks.

2.2 Design Principles

- Synthetic Images of Simple 3D Shapes: This simplifies recognition and allows a focus on reasoning skills.
- Complete and Exclusive Information: Each image contains all necessary information, preventing reliance on external knowledge.
- Minimized Question-Conditional Bias: Rejection sampling within question families helps avoid biases.
- Structured Ground-Truth Representations: Images are annotated with object positions and attributes, and questions are represented as functional programs. These programs, built from basic functions enable in-depth analysis of required reasoning abilities.
- Complex Questions Requiring Reasoning: While images are simple, the automatically generated questions are complex, testing counting, comparison, logical reasoning, and memory.

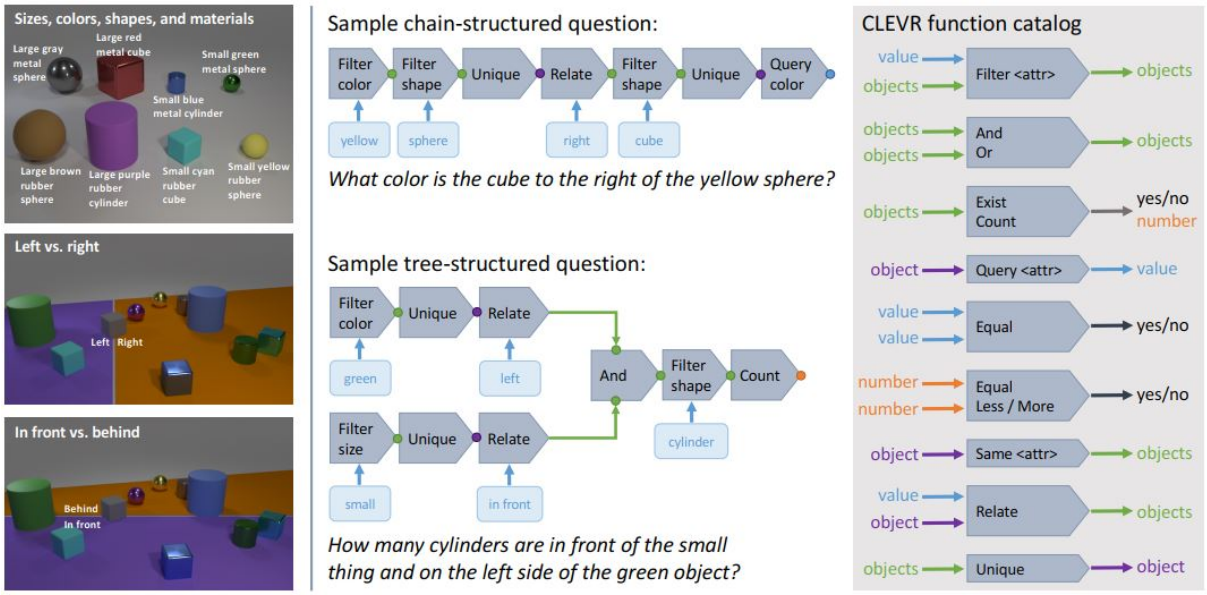


Figure 2: A field guide to the CLEVR universe. Left: Shapes, attributes, and spatial relationships. Center: Examples of questions and their associated functional programs. Right: Catalog of basic functions used to build questions.

2.3 Dataset Statistics

- CLEVR contains 100k rendered images and about one million automatically-generated questions, of which 853k are unique.
- Questions are categorized by type based on the outermost function in their program (e.g., query-color, exist, count).
- Question lengths are generally longer than in other VQA datasets.

2.4 Analysis of VQA Models on CLEVR

The CLEVR paper [1] analyzes various modern VQA systems, revealing weaknesses. For example, models struggle with:

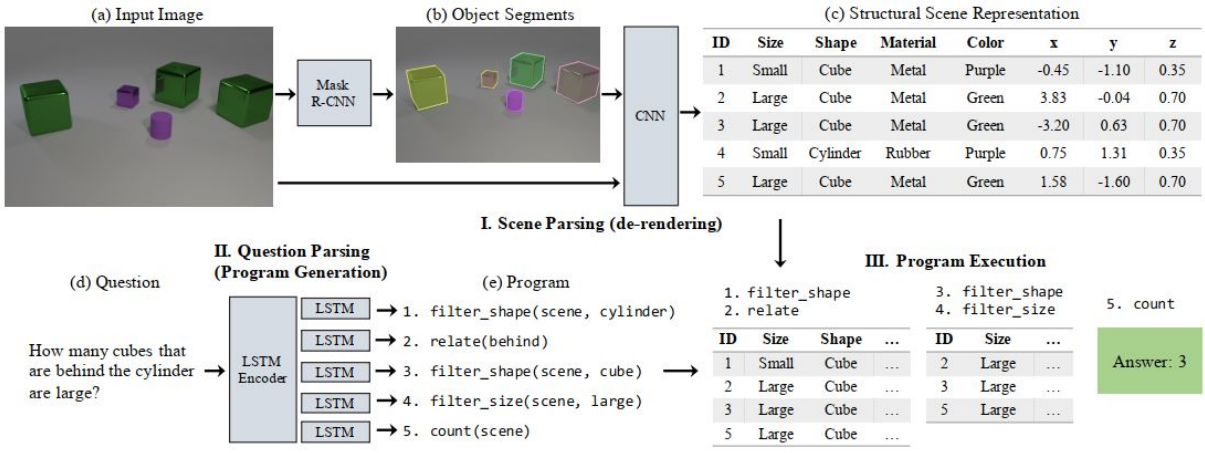
- Short-term memory: Required for attribute comparison and integer equality questions.
- Compositional reasoning: Recognizing novel attribute combinations.
- Long reasoning chains: Questions with large effective sizes and many spatial relationships.
- Spatial relationships: Often relying on absolute image position rather than true relational understanding.
- Question topology: Tree-structured questions can be more challenging than chain-structured ones.

The concept of effective question size was introduced to show that models struggle with genuinely long reasoning chains, even if shorter sub-programs can sometimes yield the correct answer.

The analysis also highlights cases where models show success, such as spatial attention aiding in identifying object attributes.

3 Core Idea: Disentangling Perception from Reasoning

The fundamental principle of Neuro-Symbolic VQA is to separate the visual and linguistic understanding from the logical reasoning process required to answer questions about images. This contrasts with end-to-end neural network approaches where these processes are implicitly learned and often intertwined. The motivation stems from the human ability to reason abstractly once a scene is perceived and a question is understood. We go deeper about this idea and explain how it is possible to disentangle perception from reasoning in the following sections.



ID	Size	Shape	Material	Color	x	y	z
1	Small	Cube	Metal	Purple	-0.45	-1.10	0.35
2	Large	Cube	Metal	Green	3.83	-0.04	0.70
3	Large	Cube	Metal	Green	-3.20	0.63	0.70
4	Small	Cylinder	Rubber	Purple	0.75	1.31	0.35
5	Large	Cube	Metal	Green	1.58	-1.60	0.70

ID	Size	Shape	...
1	Small	Cube	...
2	Large	Cube	...
3	Large	Cube	...
5	Large	Cube	...

ID	Size	...
2	Large	...
3	Large	...
5	Large	...

Figure 3: The model has three components: first, a scene parser (de-renderer) that segments an input image (a-b) and recovers a structural scene representation (c); second, a question parser (program generator) that converts a question in natural language (d) into a program (e); third, a program executor that runs the program on the structural scene representation to obtain the answer.

4 Architecture Overview

The NS-VQA system employs a modular architecture with three main components as you can see in Figure 3:

4.1 Scene Parser (De-renderer)

This module aims to create a structural, object-based representation of the visual scene.

- It begins by generating object segment proposals using Mask R-CNN. Proposals with a bounding box score less than 0.9 are discarded.
- For each segment, the network predicts categorical labels of discrete intrinsic attributes such as color, material, size, and shape.
- Each object segment, paired with the original image (resized to 224x224), is fed into a ResNet-34 network to extract spatial attributes like pose and 3D coordinates. The inclusion of the original image provides contextual information.
- The scene parsing module's object proposal network (Mask R-CNN) uses a ResNet-50 FPN backbone.
- The feature extraction network for continuous attributes is trained on the proposed object segments using mean square error as the loss function.
- Training of both networks within the scene parser is performed on 4,000 generated CLEVR images for 30,000 iterations with learning rate 0.002 a batch size of 8 for Mask R-CNN and 50 for the feature extraction network.
- The output is a disentangled, compact, and rich structural scene representation, detailing each object with its attributes and 3D coordinates.

4.2 Question Parser (Program Generator)

This module is responsible for converting the natural language question into a latent, hierarchical program.

- It utilizes an attention-based sequence-to-sequence (seq2seq) model with an encoder-decoder structure. This type of model is commonly used for mapping one sequence (the question) to another sequence (the program). The lecture slides also mention an Attention-based Bidirectional LSTM.
- The encoder is a bidirectional LSTM that processes the input question using jointly trained word embeddings.
- The decoder is another LSTM that generates the program tokens sequentially, using an attention mechanism to focus on relevant parts of the encoded question.

- The output is a sequence of tokens representing a functional program, where each token corresponds to a specific operation on the scene representation.

4.3 Program Executor

This module takes the symbolic program generated by the question parser and executes it on the structural scene representation produced by the scene parser to obtain the final answer.

- The executor is implemented as a collection of deterministic, generic functional modules in Python.
- These modules correspond one-to-one with the tokens in the program sequence.
- These modules perform purely symbolic operations on their inputs throughout the execution process.
- The program typically starts with a scene token, representing the initial scene representation, and subsequent modules operate on the output of the previous one.
- If a type mismatch occurs between the input and output of adjacent modules, an error flag is raised, and the model randomly samples an answer.
- Examples of these functional modules include set operations (e.g., filter, unique, union, intersect, count), boolean operations (e.g., equal_color, greater_than, exist), query operations (e.g., query_color, query_shape), and relation operations (e.g., relate_front, same_color).

5 Question Parser Architecture Details

5.1 Encoder (Bidirectional LSTM)

The encoder's role is to take the input question and convert it into a set of encoded vectors that capture the meaning of the question. The NS-VQA system uses a bidirectional LSTM for this purpose.

- A Long Short-Term Memory (LSTM) network is a type of recurrent neural network (RNN) capable of learning long-range dependencies in sequential data. It maintains a hidden state that is updated as it processes the input sequence.
- A bidirectional LSTM processes the input sequence in two directions: forward and backward. This allows it to consider the context of a word from both the words that precede it and the words that follow it.

The input to the encoder is a question of variable length. Each word in the question (x_i) is first converted into a word embedding ($\Phi_E(x_i)$), which is a vector representation of the word learned during training.

The forward LSTM processes the sequence from the first word to the last, producing an output (e_i^F) and a hidden vector (h_i^F) at each time step i:

$$e_i^F, h_i^F = \text{LSTM}\left(\Phi_E(x_i), h_{i-1}^F\right) \tag{1}$$

Similarly, the backward LSTM processes the sequence from the last word to the first, producing an output (e_i^B) and a hidden vector (h_i^B) at each time step i:

$$e_i^B, h_i^B = \text{LSTM}\left(\Phi_E(x_i), h_{i+1}^B\right) \tag{2}$$

The encoded vector (e_i) at each time step i is then formed by concatenating the forward and backward LSTM outputs:

$$e_i = \left[e_i^F, e_i^B\right] \tag{3}$$

5.2 Decoder (LSTM with Attention)

The decoder's role is to generate the output program sequence based on the encoded representation of the question. The NS-VQA system uses an LSTM with an attention mechanism for this.

- The decoder is also an LSTM that generates a hidden state (q_t) at each time step t, based on the previous output token (y_{t-1}) and its embedding ($\Phi_D(y_{t-1})$):

$$q_t = \text{LSTM}\left(\Phi_D(y_{t-1})\right) \tag{4}$$

- The attention mechanism allows the decoder to focus on the most relevant parts of the encoded input question when generating each token of the output program. It computes a context vector (c_t) as a weighted sum of the encoded states (e_i) from the encoder. The weights (α_{ti}) indicate the importance of each encoded state for the current decoding step.

The attention weights are calculated as follows:

$$\alpha_{ti} \propto \exp\left(q_t^T W_A e_i\right) \tag{5}$$

where W_A is an attention weight matrix. For simplicity, the paper [3] mentions setting the dimensions of q_t and e_i to be the same and W_A to be an identity matrix, which simplifies the weight calculation to:

$$\alpha_{ti} \propto \exp\left(q_t^T e_i\right) \tag{6}$$

The context vector (c_t) is then computed as the weighted sum of the encoded vectors:

$$c_t = \sum_i \alpha_{ti} e_i \tag{7}$$

Finally, the context vector (c_t) is concatenated with the decoder's hidden state (q_t) and passed through a fully connected layer with a softmax activation function to produce a probability distribution over the next token (y_t) in the output program sequence:

$$y_t \sim \text{softmax}\left(W_O\left[q_t, c_t\right]\right) \tag{8}$$

where W_O is a weight matrix for the output layer.

The encoder and decoder in the NS-VQA question parser both have two hidden layers with a 256-dimensional hidden vector, and the word vectors have a dimension of 300.

6 Training the Modules

The training process involves distinct steps for the scene parser and the question parser:

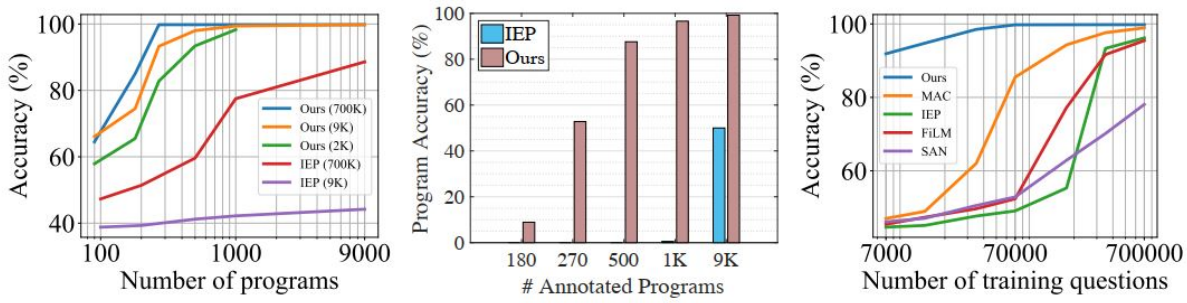
6.1 Scene Parser Training

- The Mask R-CNN is trained on rendered images and object masks. For CLEVR, since original masks weren't available, they were generated using the CLEVR dataset generation tool.
- The bounding-box classifier contains 48 classes representing combinations of intrinsic attributes.
- The feature extraction CNN is trained using segment-label pairs, where each segment is concatenated with the original image.
- Training details (iterations, learning rate, batch size) were mentioned earlier in section 4.

6.2 Question Parser Training

A two-step procedure is adopted:

- Pre-training (Supervised): The question parser is initially trained on a small number of ground truth question-program pairs. The slides mention ∼100-300 pairs for 20,000 iterations with a learning rate of 7×10^{-4}. This teaches the parser to map questions to programs. The NS-VQA paper [3] specifies using 90, 180, or 270 programs, uniformly drawn from the 90 question families of CLEVR, for pretraining.
- Post-training (Reinforcement Learning): The pre-trained parser is then fine-tuned using a larger set of question-answer pairs. The reward is 1 if the executed program gives the correct answer, and 0 otherwise. The REINFORCE algorithm is used for optimization, maximizing the expected return over a constant baseline with a decay weight of 0.9. This stage runs for at most 2 million iterations with early stopping and a learning rate of 10^{-5}. The batch size is fixed at 64 for both stages. The lecture slides highlight REINFORCE's sample efficiency (both in pre-training and post-training). You can refer to Figure 4 for more details about efficiency.



(a) Acc. vs. # pretraining programs (b) Program acc. vs. # programs (c) Acc. vs. # training data

Figure 4: The model exhibits high data efficiency while achieving state-of-the-art performance and preserving interpretability. (a) QA accuracy vs. number of programs used for pretraining; different curves indicate different numbers of question-answer pairs used in the REINFORCE stage. (c) QA accuracy vs. total number of training question-answer pairs; our model is pretrained on 270 programs.

7 REINFORCE Algorithm Details

7.1 Question Parser Training

The training process consists of two phases:

- **Pre-training**: Supervised learning with ∼100-300 question-program pairs for 20,000 iterations
- **Post-training**: Reinforcement learning with ∼9k-700k question-answer pairs for up to 2 million iterations (with early stopping)
  - Reward function: $R = \begin{cases} 1 & \text{if program execution yields correct answer} \\ 0 & \text{otherwise} \end{cases}$
  - Training algorithm: REINFORCE policy gradient method

7.2 REINFORCE Fundamentals

The REINFORCE algorithm optimizes the expected return $J(\theta)$:

$$\theta^\star = \arg\max_\theta J(\theta) = \arg\max_\theta \mathbb{E}_{\tau\sim p_\theta(\tau)}\left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t)\right] \tag{9}$$

Where the expected return can be expressed as:

$$J(\theta) = \mathbb{E}_{\tau\sim p_\theta(\tau)}[r(\tau)] = \mathbb{E}_{\tau\sim p_\theta(\tau)}[\sum_{t=1}^{T} r(\mathbf{s}_t, \mathbf{a}_t)] = \int p_\theta(\tau) r(\tau) d\tau \tag{10}$$

7.3 Gradient Derivation

The policy gradient is derived as follows:

$$\nabla_\theta J(\theta) = \int \nabla_\theta p_\theta(\tau) r(\tau) d\tau \tag{11}$$

$$= \int \frac{p_\theta(\tau)}{p_\theta(\tau)} \nabla_\theta p_\theta(\tau) r(\tau) d\tau \tag{12}$$

$$= \int p_\theta(\tau) \frac{\nabla_\theta p_\theta(\tau)}{p_\theta(\tau)} r(\tau) d\tau \tag{13}$$

$$= \int p_\theta(\tau) \nabla_\theta \log p_\theta(\tau) r(\tau) d\tau \quad \text{(Likelihood ratio trick)} \tag{14}$$

$$= \mathbb{E}_{\tau\sim p_\theta(\tau)} \left[\nabla_\theta \log p_\theta(\tau) r(\tau)\right] \tag{15}$$

7.4 Trajectory Probability

The probability of a trajectory τ (a program generated by LSTM) under policy π_θ is:

$$p_\theta(\tau) = p(\mathbf{s}_1) \prod_{t=1}^{T} \pi_\theta(\mathbf{a}_t|\mathbf{s}_t) p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t) \tag{16}$$

Taking the logarithm:

$$\log p_\theta(\tau) = \log p(\mathbf{s}_1) + \sum_{t=1}^{T} \log \pi_\theta(\mathbf{a}_t \mid \mathbf{s}_t) + \log p(\mathbf{s}_{t+1} \mid \mathbf{s}_t, \mathbf{a}_t) \tag{17}$$

7.5 Final Gradient Expression

$$\nabla_\theta J(\theta) = E_{\tau \sim p_\theta(\tau)} \left[\nabla_\theta \log p_\theta(\tau) r(\tau) \right] \tag{18}$$

$$\nabla_\theta J(\theta) = E_{\tau \sim p_\theta(\tau)} \left[\nabla_\theta \left[\log p(s_1) + \sum_{t=1}^{T} \log \pi_\theta(a_t|s_t) + \log p(s_{t+1}|s_t, a_t) \right] r(\tau) \right] \tag{19}$$

Since initial state and dynamics are independent of θ:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta(\tau)} \left[\left(\sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(\mathbf{a}_t \mid \mathbf{s}_t) \right) \left(\sum_{t=1}^{T} r(\mathbf{s}_t, \mathbf{a}_t) \right) \right] \tag{20}$$

7.6 Monte Carlo Implementation

In practice, we approximate the expectation using N samples:

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \left(\sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(\mathbf{a}_{i,t} \mid \mathbf{s}_{i,t}) \right) \left(\sum_{t=1}^{T} r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) \right) \tag{21}$$

7.7 Parameter Update

The parameters are updated via gradient ascent:

$$\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta) \tag{22}$$

Where α is the learning rate (10^{-5} in NS-VQA).

7.8 Algorithm Steps

The complete REINFORCE algorithm:

1. **Sample trajectories**: Generate programs $\{\tau^i\}$ by running current policy π_θ
2. **Compute rewards**: Execute programs to obtain rewards $r(\tau^i) = \sum_t r(s_i^t, a_i^t)$
3. **Estimate gradient**:
$$\nabla_\theta J(\theta) \approx \sum_i \left(\sum_t \nabla_\theta \log \pi_\theta(\mathbf{a}_t^i \mid \mathbf{s}_t^i) \right) \left(\sum_t r(s_i^t, a_i^t) \right)$$
4. **Update parameters**:
$$\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$$

Remark 2. *In essence, REINFORCE encourages the question parser to generate programs that lead to correct answers by increasing the probability of successful programs and decreasing the probability of unsuccessful ones, based on the reward signal received after executing the generated program. The process involves sampling programs, executing them, and then updating the parser's parameters based on the outcome. The sample efficiency of REINFORCE in both pre-training and post-training phases is a key characteristic.*

7.9 Variance Reduction

To reduce variance in gradient estimates:

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \left(\sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(\mathbf{a}_{i,t} \mid \mathbf{s}_{i,t}) \right) (r(\tau^i) - b) \tag{23}$$

Where b is a baseline (constant baseline with decay weight 0.9 in NS-VQA).

Remark 3. *The REINFORCE algorithm follows the general reinforcement learning paradigm:*

- *Generate samples by running the current policy*
- *Estimate returns from experience*
- *Improve the policy using gradient ascent*

This approach is particularly effective in NS-VQA due to the deterministic nature of program execution, which provides clear reward signals.

8 Key Advantages and Results

The neuro-symbolic approach offers several advantages:

8.1 Robustness to Long Program Traces and Complex Reasoning

Executing programs on a symbolic space is more robust to long sequences of reasoning steps compared to purely neural models. This allows the model to solve complex reasoning tasks more effectively, achieving a near-perfect accuracy of 99.8% on the CLEVR dataset when trained with 270 pretraining programs.

8.2 Data and Memory Efficiency

The model performs well even after learning on a small number of training data. Furthermore, the compact structural image representation requires significantly less storage (less than 100 bytes per image) compared to storing original images or feature maps in attention-based methods (at least 20KB per image).

8.3 Transparency and Interpretability

The use of symbolic scene representations and program traces provides full transparency to the reasoning process. Each execution step can be interpreted and diagnosed, allowing for a deeper understanding of how the model arrives at an answer. Qualitative results in Figure 6 (Figure 3 of the NS-VQA paper [3]) illustrate the model's ability to recover the correct programs.

8.4 Generalization Capabilities

- Unseen Attribute Compositions (CLEVR-CoGenT): The model demonstrates the ability to generalize to novel combinations of attributes, as evaluated on the CLEVR-CoGenT dataset (see Table 1). As described in the lecture slides, this dataset tests the model's ability to handle color-shape pairings not seen during training (split A only contains cubes that are either gray, blue, brown or yellow, and cylinders that are red, green, purple or cyan. split B has the opposite color-shape pairs for cubes and cylinders.). The NS-VQA paper [3] shows that while the vanilla model initially struggles due to biases in attribute recognition, fine-tuning the attribute recognition network on a small amount of data from the target split significantly improves generalization. Variations like NS-VQA+Gray (using grayscale images for shape recognition) and NS-VQA+Ori (image parser trained on original CLEVR) also show strong generalization across splits.
- Human Questions (CLEVR-Humans): The model generalizes well to human-generated questions on CLEVR images (CLEVR-Humans). The setup involves pretraining on a limited number of CLEVR programs and fine-tuning with REINFORCE on CLEVR-Humans (The REINFORCE stage lasts for at most 1M iterations; early stop is applied.), using fixed GloVe word embeddings. The results show that NS-VQA outperforms IEP ("Inferring and executing programs for visual reasoning" paper by Johnson et al. [2]) by a considerable



Q: How many trees are behind the farthest animal?
P: scene, filter_animal, filter_farthest, unique, relate_behind, filter_tree, count
A: 1

Q: What direction is the closest creature facing?
P: scene, filter_creature, filter_closest, unique, query_direction
A: left

Q: Are there wolves farther to the camera than the animal that is facing right?
P: scene, filter_animal, filter_face_right, unique, relate_farther, filter_wolf, exist
A: yes

(a) Sample results on the Minecraft dataset.

# Programs	Accuracy
50	71.1
100	72.4
200	86.9
500	87.3

(b) Question answering accuracy with different numbers of annotated programs

Figure 5: The model also applies to Minecraft, a world with rich and hierarchical scene context and different visual appearance.

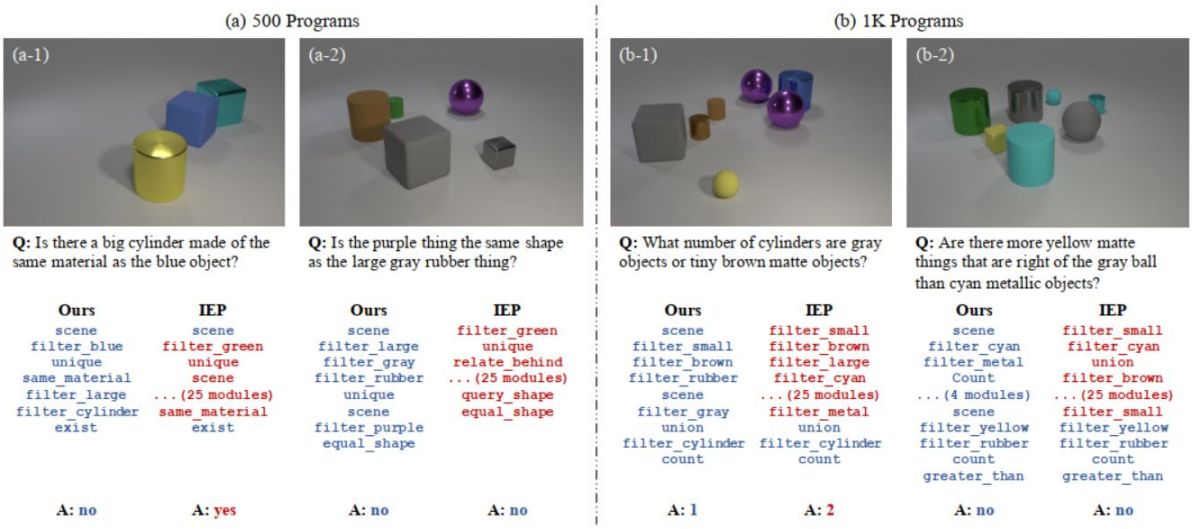


Figure 6: Qualitative results on CLEVR. Blue color indicates correct program modules and answers; red indicates wrong ones. The model is able to robustly recover the correct programs compared to the IEP baseline [2].

margin under small amounts of annotated programs as depicted in Table 2, highlighting the benefits of the structural scene representation and symbolic program executor for leveraging the strong exploration power of REINFORCE.

- New Scene Contexts (Minecraft): The framework can be extended to new visual environments with different object sets and attributes, such as Minecraft as depicted in Figure 5 5. The lecture slides and the NS-VQA paper [3] describe the adaptation to Minecraft scenes with a different structural representation (category, facing direction, larger set of 3D objects with richer image content and visual appearance) and hierarchical attributes (e.g., "wolf" and "pig" are "animals", "animal" and "tree" are "creatures"). Training follows a similar procedure, and results show promising performance, with most errors attributed to perception of occluded objects. The model uses the first 9,000 images with 88,109 questions for training and the remaining 1,000 images with 9,761 questions for testing in Minecraft setup.

Table 1: Generalization results on CLEVR-CoGenT.

Methods	Not Fine-tuned		Fine-tune	Fine-tuned	
	A	B	on	A	B
CNN + LSTM + SA	80.3	68.7	B	75.7	75.8
IEP (18K programs)	96.6	73.7	B	76.1	92.7
CNN + GRU + FILM	98.3	78.8	B	81.1	96.9
TbD + reg	98.8	75.4	B	96.9	96.3
NS - VQA (ours)	**99.8**	63.9	B	64.9	98.9
NS - VQA (ours)	**99.8**	63.9	A + B	**99.6**	**99.0**
NS - VQA + Gray (ours)	99.6	98.4	-	-	-
NS - VQA + Ori (ours)	**99.8**	**99.7**	-	-	-

Table 2: Question answering accuracy on CLEVR-Humans.

#Programs	NS-VQA	IEP
100	60.2	38.7
200	**65.2**	40.1
500	**67.8**	49.2
1K	**67.8**	63.4
18K	**67.0**	66.6

9 Conclusion and Future Directions

The NS-VQA approach represents a significant step towards more robust, interpretable, and data-efficient VQA systems. In summary, this lecture highlights a neuro-symbolic approach to VQA that explicitly separates perception and reasoning. The scene parser extracts a structured representation of the visual scene, while the question parser generates a symbolic program representing the reasoning steps. Training involves a combination of supervised learning and reinforcement learning (REINFORCE) to learn the mapping from questions to executable programs. The results on various datasets demonstrate the model's ability to generalize to novel attribute combinations, human-generated questions, and new visual environments.

Furthermore, CLEVR serves as a valuable diagnostic tool for understanding the strengths and weaknesses of such systems. Future work can explore incorporating explicit short-term memory, encouraging disentangled representations, and developing methods for learning structured representations that generalize to truly novel situations. The combination of deep representation learning with symbolic program execution offers a promising direction for advancing visually grounded language understanding.

References

[1] Justin Johnson, Bharath Hariharan, Laurens van der Maaten, Li Fei-Fei, C. Lawrence Zitnick, and Ross Girshick. CLEVR: A diagnostic dataset for compositional language and elementary visual reasoning. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.

[2] Justin Johnson, Bharath Hariharan, Laurens van der Maaten, Judy Hoffman, Li Fei-Fei, C. Lawrence Zitnick, and Ross Girshick. Inferring and executing programs for visual reasoning. In *IEEE International Conference on Computer Vision (ICCV)*, 2017.

[3] Kexin Yi, Jiajun Wu, Chuang Gan, Antonio Torralba, Pushmeet Kohli, and Joshua B. Tenenbaum. Neural-symbolic VQA: Disentangling reasoning from vision and language understanding. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018. URL `https://arxiv.org/abs/1810.02338`. arXiv:1810.02338.